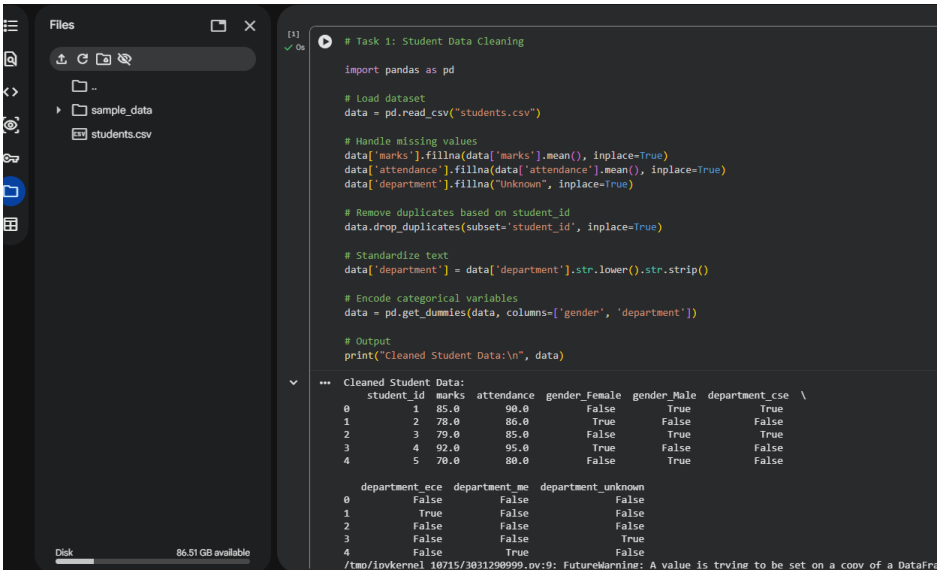
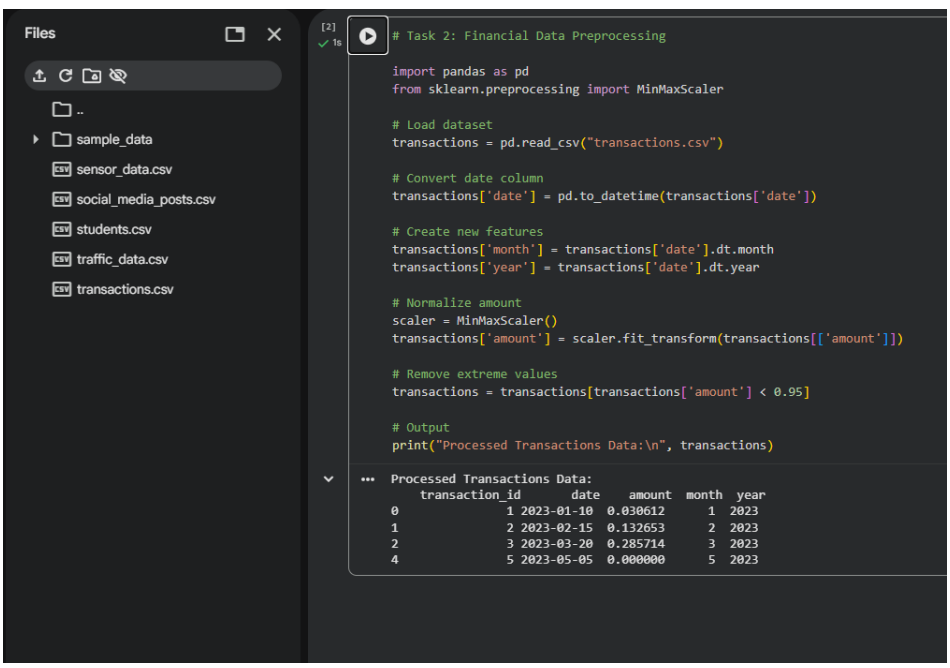
ASSIGNMENT-17.2

TASK 1: Student Academic Data Cleaning



TASK 2: Financial Transaction Data Preprocessing



TASK 3: Environmental Sensor Data Preparation

```python
# Task 3: Sensor Data Cleaning

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load dataset
data = pd.read_csv("sensor_data.csv")

# Handle missing values
data.fillna(data.mean(numeric_only=True), inplace=True)

# Encode categorical variable
data = pd.get_dummies(data, columns=['sensor_status'])

# Scale numerical features
scaler = StandardScaler()
data[['temperature','humidity','air_quality_index']] = scaler.fit_transform(
    data[['temperature','humidity','air_quality_index']]
)

# Split dataset
train, test = train_test_split(data, test_size=0.2)

# Output
print("Training Data:\n", train)
print("Testing Data:\n", test)
```

```
Training Data:
    temperature  humidity  air_quality_index  sensor_status_active  \
4     1.621266  1.700840                1.5                 False
3    -1.405098  0.000000               -1.5                  True
1     0.324253 -0.566947                0.0                 False
0    -0.540422 -1.322876               -0.5                  True

   sensor_status_inactive
4                    True
3                   False
1                    True
0                   False
Testing Data:
    temperature  humidity  air_quality_index  sensor_status_active  \
2          0.0  0.188982                0.5                  True

   sensor_status_inactive
2                   False
```

TASK 4: Smart City Traffic Data Cleaning

```python
# Task 4: Traffic Data Cleaning

import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Load dataset
traffic = pd.read_csv("traffic_data.csv")

# Standardize text fields
traffic['road'] = traffic['road'].str.lower().str.strip()
traffic['location'] = traffic['location'].str.lower().str.strip()

# Fill missing values
traffic['traffic_density'].fillna(traffic['traffic_density'].mean(), inplace=True)

# Remove duplicates
traffic.drop_duplicates(inplace=True)

# Normalize speed and vehicle_count
scaler = MinMaxScaler()
traffic[['speed','vehicle_count']] = scaler.fit_transform(
    traffic[['speed','vehicle_count']]
)

# Output summary
print("Cleaned Traffic Data Summary:\n", traffic.describe())
```

```
Cleaned Traffic Data Summary:
        traffic_density     speed  vehicle_count
count         4.000000  4.000000       4.000000
mean         85.000000  0.468750       0.383333
std          25.166115  0.413005       0.433333
min          60.000000  0.000000       0.000000
25%          75.000000  0.281250       0.150000
50%          80.000000  0.437500       0.266667
75%          90.000000  0.625000       0.500000
max         120.000000  1.000000       1.000000
/tmp/ipykernel_10715/726075566.py:14: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assig
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting value

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].meth


  traffic['traffic_density'].fillna(traffic['traffic_density'].mean(), inplace=True)
```

TASK 5: Social Media Text Cleaning & Feature Extraction

```python
# Task 5: Social Media Text Cleaning

import pandas as pd
import re
from nltk.corpus import stopwords
import nltk

# Download stopwords (run once)
nltk.download('stopwords')

# Load dataset
posts = pd.read_csv("social_media_posts.csv")

# Remove null and duplicate values
posts.dropna(subset=['text'], inplace=True)
posts.drop_duplicates(inplace=True)

# Clean text
def clean_text(text):
    text = re.sub(r"http\S+", "", text)      # remove URLs
    text = re.sub(r"[^a-zA-Z ]", "", text)   # remove special chars
    text = text.lower()                      # lowercase
    return text

posts['text'] = posts['text'].apply(clean_text)

# Tokenization
posts['tokens'] = posts['text'].apply(lambda x: x.split())

# Remove stopwords
stop_words = set(stopwords.words('english'))
posts['tokens'] = posts['tokens'].apply(
    lambda x: [word for word in x if word not in stop_words]
)

# Feature extraction
posts['word_count'] = posts['tokens'].apply(len)

# Output
print("Processed Social Media Data:\n", posts)
```

```
Processed Social Media Data:
    id                  text                     tokens  word_count
0   1   i love this product            [love, product]           2
1   2  worst experience ever  [worst, experience, ever]           3
2   3       amazing service          [amazing, service]           2
4   5       amazing service          [amazing, service]           2
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```